INTELLIGENT PROJECT MANAGEMENT SYSTEM FOR UNDERGRADUATE STUDENTS

COMMON INTEGRATED SOLUTION REPORT

Team member	Student ID	Individual component
Bandara N D M	IT22114662	Intelligent Team Formation and Topic Feasibility Analysis
Lekamge R L M D	IT22117014	Adaptive Scheduling and Generative Project Planning
Rashmika R M D	IT22101624	Automated Quality Assessment and Adaptive Mentorship
Kulathunga S A I K	IT22109576	Predictive Risk Monitoring and Contribution Analytics

Report submitted in partial fulfilment of the requirements for the

BSc (Hons) in Information Technology

Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

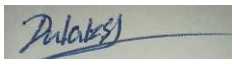
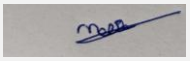
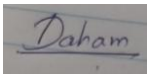
Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. We declare that this common report is our own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of our knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. We grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this report, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. We have disclosed all use of generative AI and other automated tools in accordance with the module policy. We remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this report.

Student name	Student ID	Signature	Date
Bandara N D M	IT22114662		11/10/2026
Lekamge R L M D	IT22117014		11/10/2026
Rashmika R M D	IT22101624		11/10/2026
Kulathunga S A I K	IT22109576		11/10/2026

Supervisor certification. The above candidates carried out this research project under my supervision.

Role and name	Signature	Date
Supervisor: Prof. Samantha Rajapaksha		
Co-supervisor: Ms. Kaushika Kahatapitiya		

ABSTRACT

Undergraduate research projects require coordinated decisions across team formation, project planning, proposal development, progress tracking, and academic supervision. This common report presents the Intelligent Project Management System for Undergraduate Students (IPMS), a human-supervised prototype that integrates four complementary research components: Intelligent Team Formation and Topic Feasibility Analysis, Adaptive Scheduling and Generative Project Planning, Automated Quality Assessment and Adaptive Mentorship, and Predictive Risk Monitoring and Contribution Analytics. The solution combines multi-objective optimization, proposal-grounded scheduling, machine-learning and NLP-based proposal support, and repository/task analytics within a common lifecycle-oriented application. Evaluation used component-appropriate evidence rather than a single system-wide accuracy measure. Team Formation achieved a mean hypervolume of 0.9479 and processed a synthetic 1,000-student/250-project scenario in approximately 30.15 seconds; its 60-student real-response case achieved complete modeled technical coverage and 95% preference satisfaction. Scheduling recovered 73 of 75 explicit WBS tasks in controlled PDFs, reported 81.27% held-out accuracy for a JIRA-derived proxy classifier, and later passed 6/6 focused PostgreSQL remediation tests for live LPT assignment/versioning and hall-capacity safeguards. Risk Monitoring achieved 97.63% held-out accuracy on a 4,000-record synthetic dataset and demonstrated dashboard and automated alert delivery. Proposal Assessment implements a version-aware human-in-the-loop review workflow, although its selected classifier and grading values remain performance targets requiring experimental verification. A Team Formation REST response has been locally consumed by the Scheduling team view, but complete automatic backend data exchange among all four components is not yet verified. Overall, IPMS demonstrates a coherent decision-support architecture while identifying the integration, validation, privacy, security, and deployment work required before institutional adoption.

Keywords: Intelligent project management, decision support, multi-objective optimization, adaptive scheduling, educational analytics

ACKNOWLEDGEMENTS

The research team gratefully acknowledges the guidance of Prof. Samantha Rajapaksha and the academic supervisors and staff who supported the project. The team also acknowledges the participants and project stakeholders who supported component-level requirement review, data collection, testing, and evaluation. The Proposal Assessment work benefited from approved access to the Exposía study materials made available through TU Darmstadt's TUdatalib repository. The team appreciates the cooperation of all four group members in developing, testing, documenting, and presenting the Intelligent Project Management System for Undergraduate Students.

TABLE OF CONTENTS

1 INTRODUCTION AND PROJECT CONTEXT 1

1.1 Background and context 1

1.2 Stakeholders and user needs 1

1.3 Alignment with the Sustainable Development Goals 2

1.4 Purpose and scope of the common report 3

2 OVERALL PROBLEM DEFINITION AND FOUR SUBPROBLEMS 4

2.1 Overall problem definition 4

2.2 Root causes and overall research gap 4

2.3 Subproblem and component mapping 5

2.4 Overall aim and group objectives 6

2.5 Scope assumptions and constraints 6

3 OVERALL SOLUTION STRUCTURE 8

3.1 Integrated solution overview 8

3.2 High level architecture 8

3.3 Component 1 summary 9

3.4 Component 2 summary 9

3.5 Component 3 summary 10

3.6 Component 4 summary 11

3.7 End to end user and data flow 11

4 COMPONENT INTEGRATION AND COHESIVE OPERATION 13

4.1 Integration strategy 13

4.2 Interfaces data contracts and shared models 15

4.3 Shared nonfunctional requirements 19

4.4 Integrated workflow and exception handling 21

4.5 Team contribution and dependency matrix 23
5 INTEGRATED IMPLEMENTATION AND PROJECT EXECUTION 26
5.1 Common technology stack and infrastructure 26
5.2 Development and integration process 28
5.3 Deployment and operational configuration 32
5.4 Evidence of team execution 34
6 INTEGRATED TESTING EVALUATION AND VALIDATION 36
6.1 Evaluation framework and success criteria 36
6.2 Integration and end to end testing 38
6.3 Overall results 40
6.4 Stakeholder and user validation 43
6.5 Achievement of common objectives 43
6.6 Integrated limitations and threats 45
7 COMMERCIALISATION, SUSTAINABILITY AND RESPONSIBLE INNOVATION 47
7.1 Value proposition and beneficiaries 47
7.2 Market alternatives and differentiation 48
7.3 Business model and feasibility 49
7.4 Adoption intellectual property and compliance 49
7.5 Sustainable Development Goal contribution 51
8 CONCLUSION AND FUTURE DIRECTIONS 53
8.1 Overall conclusion 53
8.2 Principal integrated contribution 53
8.3 Future directions 54
REFERENCES 56
APPENDICES 57

Appendix A Integrated interface specification 57

Appendix B Team contribution declaration 62

LIST OF FIGURES

Figure 3.1 High-level architecture and verified integration boundaries of IPMS**.....8**
Figure 3.2 Supervisor review and finalization workflow of the Automated Quality Assessment and Adaptive Mentorship component…….**10**
Figure 4.1 Team Formation active-allocation data rendered in the Scheduling team view…….**13**
Figure 5.1 Balanced Gantt schedule after LPT workload assignment in the Scheduling component……**26**
Figure 6.1 Overall mean hypervolume of the evaluated Team Formation allocation methods……...**36**
Figure 6.2 Predictive Risk Monitoring dashboard showing project risk and activity warnings……..**36**

LIST OF TABLES

Table 1.1 Stakeholders and user needs 2
Table 2.1 Subproblem and component mapping 5
Table 4.1 Current interface and integration status 14
Table 4.2 Shared identifiers and current mapping status 16
Table 4.3 Shared nonfunctional requirements and current evidence 18
Table 4.4 Integrated workflow exceptions and current handling 20
Table 4.5 Team contribution and dependency matrix 21
Table 5.1 Common technology stack and component infrastructure 24
Table 5.2 Current implementation and integration evidence 27
Table 5.3 Operational configuration and deployment boundaries 29
Table 5.4 Available evidence of team execution 30
Table 6.1 Integrated evaluation framework and success criteria 32
Table 6.2 Current integration and end-to-end verification status 34
Table 6.3 Synthesised component-level results 35
Table 6.4 Achievement of common group objectives 38
Table 7.1 Integrated value proposition and beneficiaries 40
Table 7.2 Responsible adoption and compliance readiness 43
Table A.1 Integrated interface specification and verification status 50
Table A.2 Shared identifier concepts and current mapping boundaries 51
Table B.1 Team contribution declaration. 52

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CI	Contribution Index
IPMS	Intelligent Project Management System
ML	Machine Learning
NLP	Natural Language Processing
NSGA-II	Non-dominated Sorting Genetic Algorithm II
REST	Representational State Transfer
SDG	Sustainable Development Goal
WBS	Work Breakdown Structure

1 INTRODUCTION AND PROJECT CONTEXT

1.1 Background and context

Undergraduate research and capstone projects require students and academic staff to coordinate a sequence of interdependent decisions: forming technically suitable teams, assigning projects, converting proposals into actionable plans, reviewing proposal quality, tracking progress, and identifying contribution or delivery risks. Research on AI-enabled project management shows increasing use of data-driven and intelligent methods to support planning, monitoring, decision-making, and resource management [1]. In an academic setting, however, these activities must remain transparent and accountable because automated recommendations affect students, supervisors, and academic administration.

The Intelligent Project Management System for Undergraduate Students (IPMS) addresses this lifecycle as a coordinated decision-support problem rather than as a single isolated optimization or monitoring task. Its four research components focus on complementary stages: team/project allocation, adaptive planning and scheduling, research-proposal assessment and mentorship, and execution-stage risk and contribution monitoring. Prior work supports the relevance of computational team formation [2], semantic text representations for document analysis [3], and repository analytics for understanding software-development activity while also warning against simplistic interpretation of repository metrics [4]. These strands motivate an integrated academic project-management environment in which automated analysis assists, rather than replaces, academic judgement.

1.2 Stakeholders and user needs

The primary stakeholders are undergraduate project students, research supervisors, project coordinators, academic evaluators, and administrative staff responsible for project delivery. Their needs differ across the project lifecycle but share a requirement for consistent, traceable, and reviewable information. Table 1.1 summarizes the principal stakeholder needs addressed by the integrated solution.

Across the components, human oversight remains a common design principle. Team Formation requires staff selection of the final allocation; Scheduling treats generated

plans and activity evidence as reviewable suggestions; Proposal Assessment keeps supervisor review as the academic decision boundary; and Risk Monitoring uses contribution and risk indicators as monitoring support rather than as the sole basis for grading.

Stakeholder	Primary need	IPMS support
Students	Fair project/team allocation, clear plans, useful proposal guidance, transparent progress information	Preference-aware allocation, task plans, proposal feedback support, contribution/activity views
Supervisors	Reliable group context, reviewable plans/evidence, early visibility of issues	Confirmed team data, Gantt/progress evidence, proposal review workflow, risk/contribution indicators
Project coordinators / academic staff	Scalable cohort-level administration and traceable decisions	Team/project allocation, supervisor assignment, academic event scheduling, allocation history and reporting
Evaluators / institution	Consistent evidence, human oversight, privacy-aware decision support	Versioned records, dashboards, reports, advisory analytics and documented limitations

Table 1.1: Stakeholders and user needs

1.3 Alignment with the Sustainable Development Goals

The strongest common alignment is with Sustainable Development Goal 4 (Quality Education). IPMS is intended to support more structured management of undergraduate project work by improving the transparency of team allocation, planning, feedback, supervision, and monitoring. This is an intended contribution rather than a measured SDG impact: the current research does not demonstrate institution-wide improvements in learning outcomes, completion rates, or educational equity.

A secondary intended alignment with SDG 9 (Industry, Innovation and Infrastructure) may be discussed in relation to the development of an intelligent software platform and the use of modern project-management analytics. However, the common report should avoid presenting this as a demonstrated societal impact unless stronger deployment evidence is later obtained.

1.4 Purpose and scope of the common report

This common report presents IPMS as one integrated research project. It focuses on the overall problem, the relationship among the four subproblems, the common system structure, interfaces, implementation evidence, integrated evaluation, responsible innovation, and shared project contribution. Detailed mathematical formulations, model development, datasets, component experiments, and implementation evidence remain in the four individual theses.

The common report therefore does not repeat each individual thesis. Instead, it synthesizes the evidence necessary to explain why the four components belong to one project and how they collectively support the undergraduate project lifecycle. Where cross-component execution or common deployment evidence is still incomplete, the report marks that limitation explicitly rather than assuming full integration.

2 OVERALL PROBLEM DEFINITION AND FOUR SUBPROBLEMS

2.1 Overall problem definition

The common problem addressed by this research is the fragmented and largely manual management of undergraduate group research projects across team formation, project planning, proposal review, progress monitoring, and contribution assessment. These activities are interdependent, yet they are often handled through separate documents, meetings, spreadsheets, repositories, and staff judgement. This fragmentation can make it difficult to preserve consistent project context, expose trade-offs, track evidence, and identify emerging issues in a timely and transparent manner.

The problem is not simply the absence of one software application. It is the lack of a coordinated decision-support approach that can assist different academic roles at different stages while maintaining human oversight and respecting the limitations of automated models and student-generated data.

2.2 Root causes and overall research gap

Four recurring causes motivate the integrated research. First, team and project allocation requires balancing technical project needs with student preferences, creating a combinatorial decision problem. Second, a proposal does not automatically provide a dated and auditable execution plan, requiring task extraction, effort estimation, dependency handling, workload assignment, and academic-event coordination. Third, proposal review requires more than structural completeness; students and supervisors need contextual guidance on possible weaknesses and revisions. Fourth, progress meetings and raw repository counts may not adequately expose uneven contribution, inactivity, overdue work, or delivery risk.

Existing research addresses parts of these issues through educational team-formation algorithms, project-management analytics, NLP-based document analysis, and software-repository mining. The overarching gap for this project is the absence of a single undergraduate-focused prototype that deliberately covers these four stages while preserving academic review and providing traceable component-level

evidence. The present implementation achieves a common application and selected API-level connections, but complete automatic data exchange among all four components is not yet demonstrated; this distinction is retained throughout the report.

2.3 Subproblem and component mapping

Table 2.1 maps the four research subproblems to their owners and principal contribution to the complete IPMS.

The subproblems are intentionally distinct but sequentially related. Team Formation establishes the group/project context; Planning and Scheduling converts the project into an executable plan; Proposal Assessment supports research-document quality and supervisor review; and Risk/Contribution Monitoring supports execution-stage oversight. The project does not claim that every transition is currently automated at the backend level. The mapping describes the intended lifecycle and the verified responsibilities of each component.

Component / owner	**Subproblem**	**Core approach**	**Principal output**
Team Formation and Topic Feasibility — Bandara N D M	How to create feasible teams and project assignments while keeping technical suitability and student preference as separate criteria	Heuristic-Seeded NSGA-II V3 with explicit project requirements and Pareto alternatives	Staff-selectable team/project allocations and confirmed team/supervisor structure
Adaptive Scheduling and Generative Project Planning — Lekamge R L M D	How to convert project proposals into dated plans, distribute work, track evidence, and coordinate academic events	WBS extraction, effort/dependency planning, live LPT assignment, evidence review, capacity-aware academic scheduling	Versioned Gantt schedules, balanced task ownership, review evidence and academic-event drafts
Automated Quality Assessment and Adaptive Mentorship — Rashmika R M D	How to identify potential proposal weaknesses and support revision without replacing supervisor judgement	Structural checks, sentence embeddings + Logistic Regression, TF-IDF + Ridge, feedback retrieval and resource recommendations	Proposal-analysis outputs, review context, mentorship suggestions and advisory score
Predictive Risk	How to monitor	Weighted	Contribution,

Monitoring and Contribution Analytics — Kulathunga S A I K	project execution and contribution using repository/task activity without relying only on manual reports or raw commit counts	Contribution Index, 12-feature risk classification, GitHub/Jira analytics and dashboard alerts	progress, activity and High/Medium/Low risk indicators

Table 2.1: Subproblem and component mapping

2.4 Overall aim and group objectives

The common aim is to develop and evaluate an intelligent, human-supervised project-management system that supports key stages of undergraduate group research projects from team/project allocation through planning, proposal improvement, and execution monitoring.

O1. Generate feasible team/project allocations that explicitly consider project technical requirements and student project preferences.

O2. Transform project proposals into reviewable, versioned plans and schedules while supporting workload assignment, progress evidence, and academic-event coordination.

O3. Provide advisory proposal-analysis and mentorship support through structural, semantic, retrieval, and grading methods while retaining supervisor review.

O4. Provide contribution and project-risk indicators using repository and optional task-management activity for execution-stage oversight.

O5. Present the four components through a cohesive IPMS application and define stable interfaces and shared identifiers where cross-component exchange is implemented.

O6. Evaluate each component using appropriate research metrics and synthesize the evidence at system level without overstating synthetic, prototype, or unverified results.

2.5 Scope assumptions and constraints

IPMS is evaluated as a research prototype rather than as a production institutional service. The components use different datasets and evidence types, including controlled/synthetic experiments, real-response survey data, historical software-

project records, external GitHub/Jira activity, and local prototype tests. Results are therefore interpreted within their individual evaluation boundaries rather than combined into one universal accuracy or effectiveness score.

3 OVERALL SOLUTION STRUCTURE

3.1 Integrated solution overview

IPMS is structured as a lifecycle-oriented decision-support environment for undergraduate research projects. Rather than attempting to solve all project-management decisions with one model, it assigns a specialized method to each stage and brings the outputs together through a common application and defined interfaces. The central value proposition is traceability: staff can move from a confirmed team/project structure to a project plan, proposal-review support, and execution monitoring while retaining human review at key decision points.

The current implementation should be understood as a partially integrated prototype. A common application provides access to the four components; Team Formation exposes stable REST endpoints for the latest confirmed allocation; Scheduling has locally demonstrated consumption of the Team Formation active-allocation endpoint for team-view rendering; and the Proposal Assessment and Risk components operate independently within the shared system context. Complete automatic backend handoff across all four components was not established in the final evidence set and is treated as an integration limitation rather than an implemented capability.

3.2 High level architecture

The high-level architecture consists of a common user-facing IPMS layer with four component services and component-specific persistence or external data sources. Academic staff and supervisors interact with the system through web interfaces. Team Formation uses a React/FastAPI/PostgreSQL stack and exposes confirmed allocation data through REST. Scheduling uses FastAPI/PostgreSQL and integrates proposal files, progress evidence, GitHub webhooks, and academic scheduling data. Proposal Assessment uses React/FastAPI/PostgreSQL with local NLP/ML models and optional external services. Risk Monitoring uses FastAPI-based analytics with GitHub/Jira integrations, local model artefacts, dashboard interfaces, and persistence for project/user/risk-alert functions. Figure 3.1 summarises these component boundaries and distinguishes verified interfaces from lifecycle relationships for which direct data exchange has not been demonstrated.

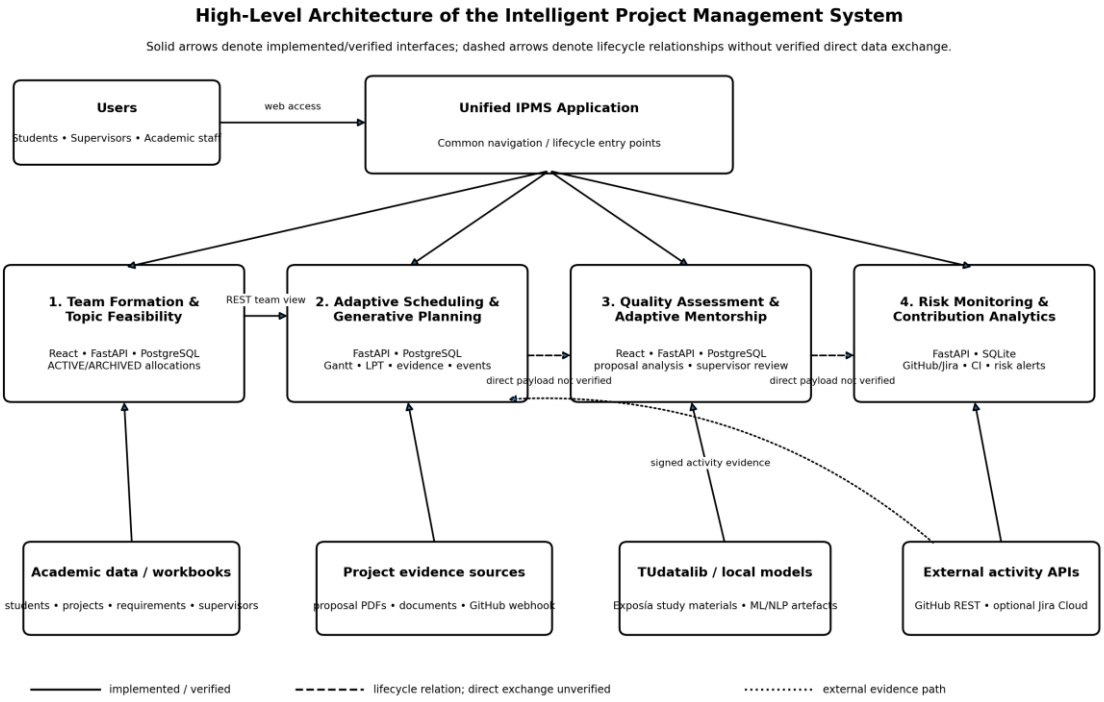


Figure 3.1: High-level architecture and verified integration boundaries of IPMS

3.3 Component 1 summary

Bandara N D M (IT22114662) developed the Intelligent Team Formation and Topic Feasibility Analysis component. It addresses the combined problem of forming student teams and assigning projects while keeping technical project suitability and student project preference as separate objectives. Inputs include self-reported competency ratings, ranked project preferences, approved projects, explicit technical requirements, team size, and supervisor information. Heuristic-Seeded NSGA-II V3, based on the established NSGA-II framework [5] generates nondominated alternatives using Technical Requirement Deficit and Preference Dissatisfaction. Academic staff review the alternatives, inspect requirement-level technical coverage, allocate supervisors, and finalize the official allocation. The resulting ACTIVE allocation provides the team/project/supervisor foundation for downstream IPMS use.

3.4 Component 2 summary

Lekamge R L M D (IT22117014) developed Adaptive Scheduling and Generative Project Planning. The component addresses the gap between a project proposal and

an executable, evidence-linked plan. It accepts a proposal, group/project parameters, member identifiers, and staff/hall availability. Rule-based WBS extraction, provisional effort estimation, dependency-aware date generation, live LPT workload assignment, document/GitHub evidence handling, and capacity-aware academic-event scheduling are used to produce versioned Gantt plans, balanced task ownership, review evidence, and Proposal/PP1/PP2/Viva event drafts. Historical JIRA classifiers and hybrid allocation experiments are treated as offline research evidence rather than as validated live student-suitability models. XGBoost is used for evaluated predictive components [6], while LPT provides an interpretable workload-balancing baseline [7].

3.5 Component 3 summary

Rashmika R M D (IT22101624) developed the Automated Quality Assessment and Adaptive Mentorship component. It addresses the difficulty of identifying substantive proposal weaknesses and providing revision support beyond simple structural checks. The component accepts proposal PDFs and uses document-structure validation, sentence embeddings with Logistic Regression, TF-IDF with Ridge Regression, historical-feedback retrieval, concept analysis, and learning-resource recommendations. Outputs include structural warnings, predicted weakness categories, related historical feedback, advisory grading information, proposal-version records, supervisor review context, and feedback artefacts. Achieved performance for the selected MiniLM/Logistic Regression and TF-IDF/Ridge models was not available in the final evidence set; the reported values are therefore treated as performance targets rather than measured results. [8].

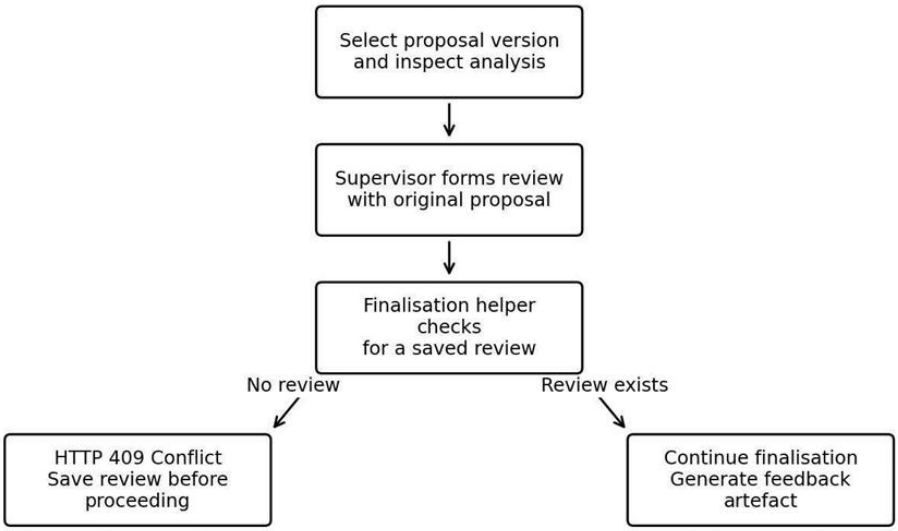


Figure 3.2: Supervisor review and finalization workflow of the Automated Quality Assessment and Adaptive Mentorship component

3.6 Component 4 summary

Kulathunga S A I K (IT22109576) developed the Predictive Risk Monitoring and Contribution Analytics component. It addresses execution-stage visibility by combining GitHub activity, optional Jira task information, weighted contribution measurement, and risk classification. The operational Contribution Index uses commit activity, task completion, collaboration, issue resolution, and code-quality indicators. A 12-feature classifier produces High, Medium, or Low risk categories, while dashboards display contribution, progress, activity, and warning indicators. The reported 97.63% Logistic Regression accuracy and approximately 0.98 weighted F1 are results on a 4,000-record synthetic dataset and are not evidence of predictive accuracy on real student project outcomes. The prototype has also demonstrated GitHub/Jira retrieval and automated high-risk email notification. AHP is examined as a multi-criteria weighting method within the contribution-analysis research [9].

3.7 End to end user and data flow

At lifecycle level, IPMS begins with academic staff preparing cohort and project information for Team Formation. After validation and multi-objective optimization, staff select and finalize a team/project allocation and allocate supervisors. The latest confirmed allocation can be retrieved through GET /api/final-

allocations/active/teams; a local trace has demonstrated that the Scheduling team-view frontend can consume and render this payload, although automatic mapping into Scheduling backend records is not yet verified.

For planning, staff provide a project proposal and scheduling parameters. The Scheduling component extracts source-linked tasks, creates a dated schedule, and can produce a balanced task-assignment version. During project execution, evidence can be attached to tasks and supervisors can review submitted documents; GitHub activity can be linked through signed webhooks. Proposal Assessment operates on proposal documents and version-associated review records to provide advisory analysis and mentorship outputs. Risk Monitoring independently retrieves GitHub and optional Jira activity to calculate contribution and risk indicators for dashboards and alerts.

This end-to-end description represents the common project lifecycle and the verified responsibilities of each component. It must not be interpreted as evidence that every data object automatically traverses all four backends. The implemented prototype combines common navigation, selected API-level integration, and independently operating component analytics. A single canonical shared-ID contract and complete cross-component transactions were not established in the final evidence set.

4 COMPONENT INTEGRATION AND COHESIVE OPERATION

4.1 Integration strategy

The integration strategy follows a modular service approach rather than a single shared codebase or database. Each research component retains responsibility for its own algorithms, persistence and local validation, while the common IPMS application provides a unified access point and selected HTTP interfaces are used where cross-component exchange has been implemented. This approach preserves component independence and allows each research method to be evaluated on its own evidence while still supporting a common project-management lifecycle.

The strongest verified cross-component boundary is provided by Team Formation. A confirmed allocation is exposed through REST endpoints that resolve the latest ACTIVE allocation, avoiding direct access to the Team Formation PostgreSQL database. Scheduling has locally demonstrated an HTTP 200 call to the active-team endpoint and rendered the returned teams in its team-view interface. This establishes API-level compatibility for display, but it does not prove that Team Formation identifiers are automatically imported into Scheduling project or academic event records.

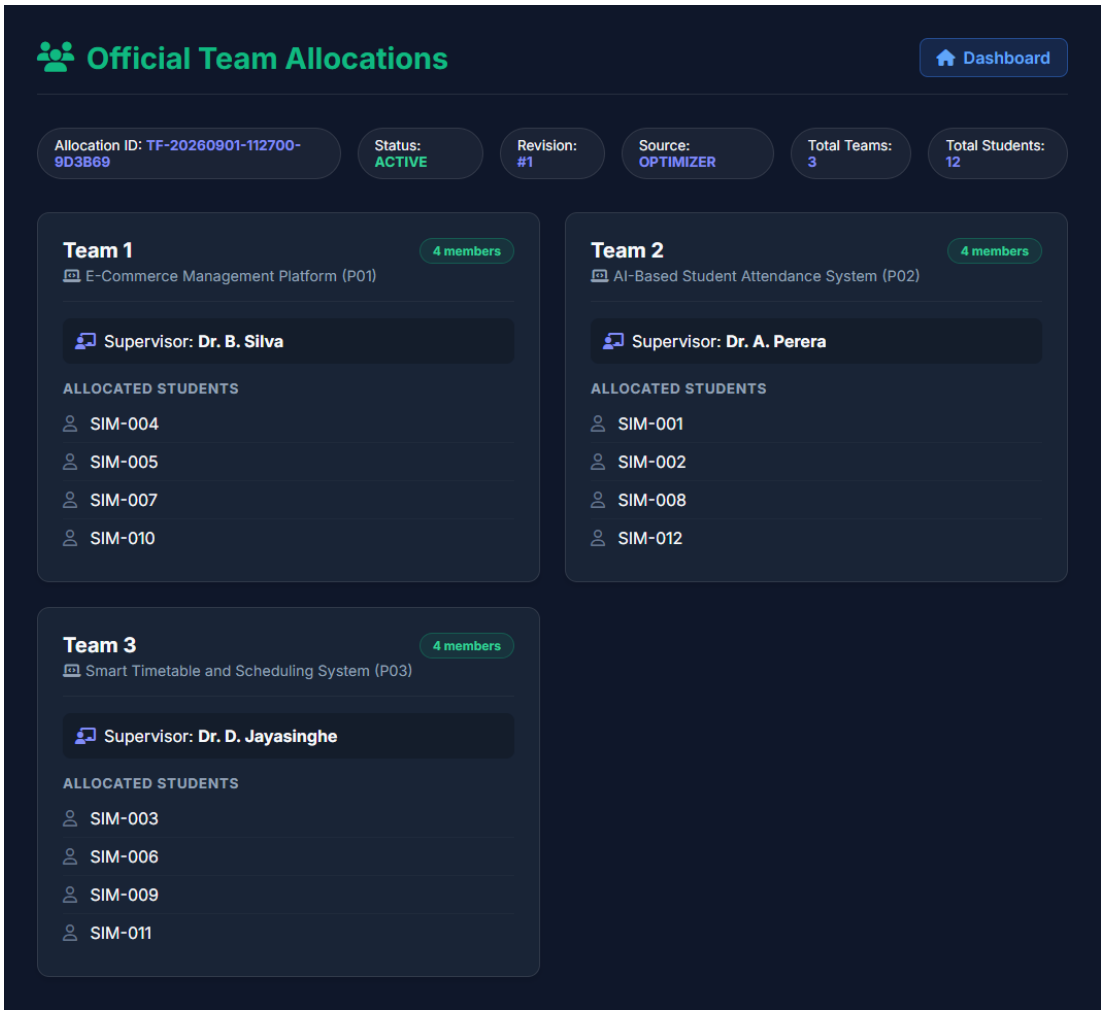


Figure 4.1: Team Formation active allocation data rendered in the Scheduling team view

The other two components are currently integrated primarily at application and lifecycle level. Proposal Assessment has a documented internal React/FastAPI/PostgreSQL workflow, but successful cross-component transactions with the other IPMS services have not been verified. Risk Monitoring is accessible within the unified IPMS application and independently retrieves GitHub and optional Jira activity, but its final handoff explicitly states that direct data exchange between all four components is not part of the current implementation. Accordingly, the common report treats integration as a combination of common application access, selected verified API exchange, and component-local analytics rather than as a fully automated four-backend pipeline.

4.2 Interfaces data contracts and shared models

Interfaces are documented according to what is currently verified, partially verified, or component-local. The principal confirmed cross-component contract is the Team Formation active-allocation API. Other endpoints listed below are important internal service boundaries but should not be described as cross-component integrations unless a producer-consumer trace is available.

Table 4.1: Current interface and integration status

Producer / Service	Interface	Consumer / Purpose	Status
Team Formation	GET /api/final-allocations/active/teams	Scheduling team-view and other downstream consumers; returns current official teams, projects, members and supervisor information	Implemented provider; local Team Formation → Scheduling view trace captured
Team Formation	GET /api/final-allocations/active/supervisors/{supervisor_id}/groups	Supervisor-oriented consumers; returns the	Implemented provider endpoint

		current groups assigned to one supervisor	
Scheduling	POST /api/schedule/upload-pdf; POST /api/schedule/generate; GET /api/schedule/history	Component-local proposal-to-plan and schedule-version workflow	Implemented component interface
Scheduling	POST /api/progress/auto-assign	Authenticated LPT task ownership and balanced schedule version	Focused PostgreSQL regression tested
Scheduling	GET /api/integrations/projects/{project_id}/document-tasks; POST /api/integrations/document-submissions	Test submission portal → Scheduling task/evidence workflow	Routes implemented; task-list GET captured. A fresh end-to-end upload trace was not retained in the final evidence set.
Scheduling	POST /api/webhooks/github	Signed GitHub task-	Route implemented with signature/repository/

		activity ingestion	task-key checks. A fresh signed delivery trace was not retained in the final evidence set.
Proposal Assessment	POST /analyze	Component-local multi-stage proposal analysis	Documented component route. Exact request/response schema and successful complete runtime verification were not available in the final evidence set.
Proposal Assessment	Review-draft / finalisation routes	Version-aware supervisor review and feedback generation	Version-aware review/finalisation behavior is documented, but exact final URLs/schemas/authentication and a complete runtime flow were not available in the final evidence set.
Risk Monitoring	GET /projects; POST /projects/upload; GET/POST /contribution; GET /dashboard; GET /api/dashboard-data; GET /api/jira-	Component-local project setup,	HTTP methods are documented in the final individual thesis; complete

	metrics; POST /predict; GET/POST /api/projects; POST /api/test-email	contributio n/risk analytics and dashboard access	request/response schemas were not consolidated for the common report.

The components do not yet share one canonical persistence schema. Instead, they use overlapping identifiers that must be reconciled at interface boundaries. Team Formation exposes allocation-level and team-level identifiers; Scheduling uses group, member, task, schedule-version and event identifiers; Proposal Assessment uses proposal/version/review identities; and Risk Monitoring uses project, student, repository and Jira identities. A common group-level identifier contract is therefore a remaining integration task.

Table 4.2: Shared identifiers and current mapping status

Identifier / concept	**Team Formation**	**Scheduling**	**Proposal Assessment**	**Risk Monitoring**	**Current status**
Student / member ID	Confirmed in members[]	Confirmed member/student ID	Not confirmed as IPMS-wide field	Confirmed student/project-member context	Shared concept exists; one frozen IPMS-wide field/schema was not established.
Team / group	team_number and group_key	group_id (e.g., R26-IT-095 form)	Not confirmed	Project/team context present	Automatic Team Formation group_key → Scheduling

					group_id mapping not verified
Project	project_id and project_title	Project/group records	Proposal/version context, exact IPMS project mapping not confirmed	Project ID/name; GitHub repository; optional Jira key	Common project concept exists; a canonical cross-component project mapping was not established.
Supervisor	supervisor_id and supervisor_name	Supervisor assignment and scheduling records	Supervisor association in review workflow	Supervisor/dashboard user context	Shared role concept exists; one common identifier/authentication schema was not established.
Version / revision	allocation_id, revision_number, ACTIVE/ARCHIVED	schedule version	proposal version / review draft	Not a common version model	Component-specific versioning

.

4.3 Shared nonfunctional requirements

Because IPMS is a research prototype composed of heterogeneous services, shared nonfunctional requirements are expressed as cross-cutting design goals rather than as fully production-certified guarantees. The current evidence supports several common principles: validation before academic state changes, human review for consequential

decisions, isolation of component databases, explicit versioning where records evolve, and transparent qualification of model outputs.

Table 4.3: Shared nonfunctional requirements and current evidence

Requirement	Common expectation	Current evidence / limitation
Reliability and data integrity	Reject invalid inputs and preserve traceable state changes	Team Formation performs extensive workbook/allocation validation and ACTIVE/ARCHIVED revisioning; Scheduling focused tests cover assignment versioning and hall-capacity rejection; Proposal runtime verification remains incomplete
Performance and scalability	Provide interactive response times for academic workflows and document large-scale limits	Team Formation processed a synthetic 1,000-student/250-project scenario in about 30.15 s; Risk uses a 120-second cache; common-system performance has not been benchmarked
Security and authentication	Protect project, student and review data with role-appropriate access	Scheduling protects live assignment and rejects demo credentials in production mode; Risk implements user authentication; Proposal cross-supervisor access requires verification; no common SSO model is confirmed
Privacy and responsible AI	Minimize sensitive data exposure and keep automated outputs advisory	Team Formation avoids unrelated personal attributes in f1/f2; Proposal scores/classifications require supervisor judgement; Risk CI/risk must not be sole grading evidence. No consolidated group-level retention/privacy policy was supplied, so this remains an

		institutional-deployment requirement.
Maintainability	Keep component responsibilities and interfaces explicit	Separate services/databases, REST boundaries and versioned records reduce coupling. A common API/version-governance process was not established in the final evidence set.
Usability and accessibility	Support staff/student workflows with clear evidence and explanations	Component interfaces exist, but formal common usability/accessibility studies have not been conducted
Recoverability / operations	Define configuration, backup, restore and degraded-mode behaviour	Risk supports GitHub-only operation when Jira is unavailable and Scheduling has component-level fallback/error handling. No group-wide backup/recovery or production-operations procedure was established.

4.4 Integrated workflow and exception handling

The normal integrated workflow begins with staff preparing cohort and approved-project information for Team Formation. After validation, the multi-objective optimizer generates alternatives and staff confirm an official allocation. The ACTIVE allocation can then be retrieved through the Team Formation REST interface. Scheduling may display this allocation and independently creates proposal-grounded task plans, balanced LPT assignment versions, evidence records and academic-event schedules. Proposal Assessment analyzes proposal documents within its own version-aware review workflow, while Risk Monitoring independently retrieves execution activity from GitHub and optional Jira to generate contribution and risk indicators. This sequence represents a common project lifecycle; it does not imply that every record is automatically propagated through every backend.

Exception handling is deliberately component-specific because the services have different inputs and failure modes. The common integration principle is to fail explicitly, preserve prior confirmed state where possible, and avoid converting missing evidence into an automatic academic decision.

Table 4.4: Integrated workflow exceptions and current handling

Area / failure	Current handling	Evidence boundary
Invalid Team Formation workbook or allocation	Reject duplicates, missing/out-of-range competencies, unknown projects, invalid requirements/preferences/supervisors and infeasible team/project counts before confirmation	Implemented validation; manual revisions are recalculated by backend before becoming ACTIVE
Team Formation revision	Previous ACTIVE allocation becomes ARCHIVED; revised confirmed record becomes ACTIVE	Implemented lifecycle; downstream /active/... endpoint remains stable
Scheduling stale/repeat assignment or invalid capacity	Focused tests reject stale/repeat ownership requests and undersized halls during generation/reschedule/publication	6/6 focused PostgreSQL regression; not a complete system test
Scheduling service unavailable to test portal	Portal can return HTTP 503; task/evidence path depends on Scheduling service availability	Component-local behavior
Invalid GitHub webhook	Signature/repository/task-key checks are present	Route and validation logic are present; a fresh

		signed webhook delivery trace was not retained in the final evidence set.
Proposal finalisation without saved review	Documented helper returns HTTP 409 Conflict	The saved-review precondition is documented in source/workflow evidence; a complete successful finalisation run was not verified in the final evidence set.
Proposal backend dependency/runtime failure	Previous test collection blocked by missing cachetools	Successful rerun not yet supplied
Jira unavailable for Risk Monitoring	Operate in GitHub-only mode; rate-limit messages and 120-second cache documented	Prototype behavior; formal availability testing not reported
Cross-component identifier mismatch	Do not assume automatic handoff; use verified view-level integration or component-local records until mapping is proven	Observed Team Formation sample IDs did not map automatically to Scheduling group R26-IT-095

4.5 Team contribution and dependency matrix

The division of work follows the four research components, with common architecture, report preparation, integration planning and presentation activities treated as shared work. The table below records the evidence-supported component

responsibilities and group-level dependencies. Appendix B provides the final contribution declaration for administrative confirmation

Table 4.5: Team contribution and dependency matrix

Member	Primary component responsibility	Research / development contribution	Key dependency on other components	Approval status
Bandara N D M (IT22114662)	Intelligent Team Formation and Topic Feasibility Analysis	Multi-objective formulation; NSGA-II V3; validation; technical coverage; supervisor allocation; persistence; REST APIs.	Provides the official team/project/supervisor structure for later lifecycle stages.	Individual evidence strong; shared-work wording needs group approval.
Lekamge R L M D (IT22117014)	Adaptive Scheduling and Generative Project Planning	WBS extraction; effort/dependency-aware Gantt; live LPT balancing; evidence/review workflow; academic-event scheduling.	Can consume Team Formation at view level; automatic backend import is not verified.	Component work documented; team-approved personal/shared statement not available in the final evidence set.
Rashmika R M D (IT22101624).	Automated Quality Assessment and Adaptive Mentorship	Semantic weakness analysis; advisory grading; historical-feedback retrieval; learning resources; proposal-version/review workflow.	Works within the common lifecycle; cross-component data exchange is not verified.	Candidate statement supplied; shared/integration work needs group confirmation.
Kulathunga S A I K (IT22109576)	Predictive Risk Monitoring and Contribution	Contribution Index; 12-feature risk classification; GitHub/Jira	Uses project/member/repository context; direct exchange with other backends is not	Individual contribution documented; shared activities

	Analytics	analytics; dashboard; alert/email evidence.	implemented.	recorded at group level in Appendix B

Shared work includes common IPMS framing, common application/navigation, integration planning, common-report preparation, presentations and agreed system-level decisions. These activities depend on consistent naming, identifiers, interface boundaries and evidence qualification across all four components. The final component-level contribution declaration is provided in Appendix B for administrative confirmation.

5 INTEGRATED IMPLEMENTATION AND PROJECT EXECUTION

5.1 Common technology stack and infrastructure

The complete IPMS is implemented as a set of research components that share a common project context and user-facing application boundary while retaining component-specific services and persistence. The evidence currently supports a componentised architecture rather than a single monolithic backend or shared database. Python and FastAPI are used across the four components for service logic, while web interfaces expose staff, supervisor and student workflows. REST interfaces and shared identifiers are preferred where data must cross a component boundary; direct database coupling is not treated as the integration mechanism.

The technology choices reflect the different research problems. Team Formation requires evolutionary optimisation and spreadsheet-based cohort preparation; Scheduling requires proposal parsing, effort estimation, dependency-aware planning, evidence handling and academic-event scheduling; Proposal Assessment requires NLP/ML analysis, proposal-version persistence and supervisor review; and Risk Monitoring requires external repository/task data, contribution scoring and risk classification. Consequently, persistence and frontend technologies are not identical across components, but each component exposes a clearly bounded application service.

Table 5.1: Common technology stack and component infrastructure

Component	User interface	Backend / analytics	Persistence	Key supporting technologies
Team Formation and Topic Feasibility	React + Vite staff workflow	Python, FastAPI; DEAP/custom NSGA-II services	PostgreSQL	Pydantic, OpenPyXL, Axios, Recharts; Excel cohort/project inputs; REST final-allocation APIs
Adaptive Scheduling and Generative Project Planning	Staff scheduling UI plus separate student submission portal	Python, FastAPI; WBS extraction, XGBoost effort model, DAG scheduling, LPT assignment	PostgreSQL	PDF processing, GitHub HMAC webhooks, evidence review, academic-event scheduling, calendar/Excel export

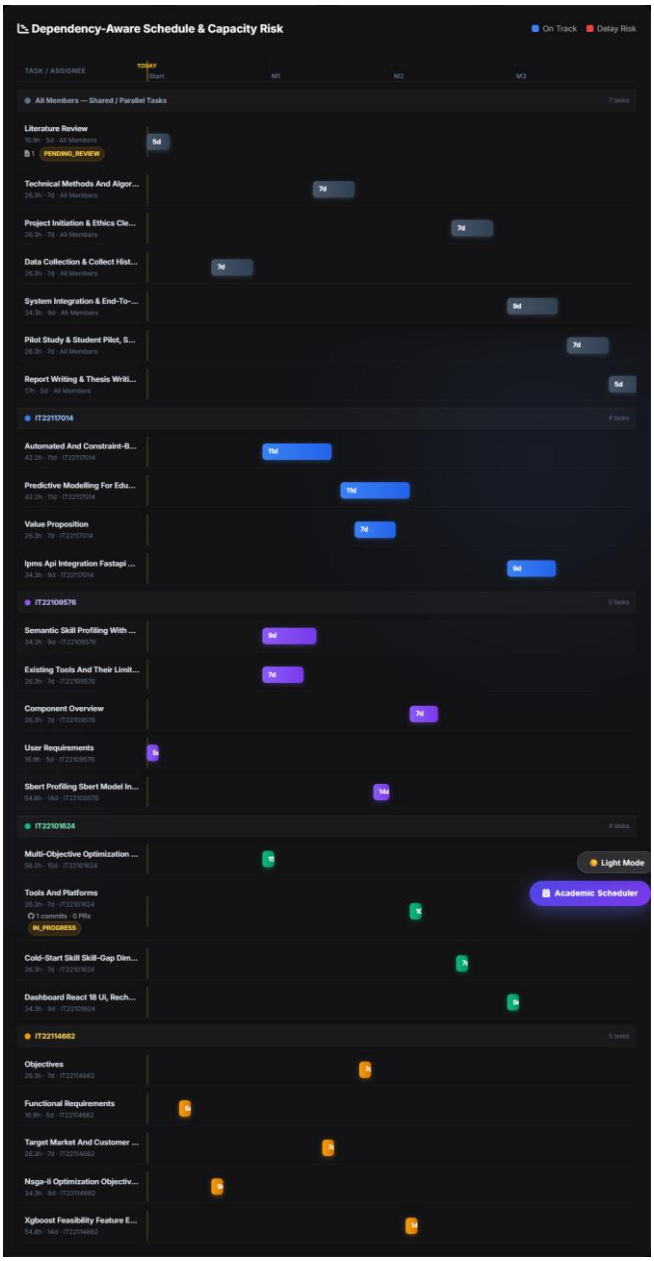
Automated Quality Assessment and Adaptive Mentorship	React 19 + Vite	FastAPI, Pydantic; sentence-transformers, scikit-learn, spaCy	PostgreSQL + SQLAlchemy	pdfjs-dist, ReportLab, feedback retrieval; optional OpenAI-compatible LLM and email service
Predictive Risk Monitoring and Contribution Analytics	HTML/CSS/Jinja2 dashboard with React/Vite source also documented	Python, FastAPI; contribution analytics and scikit-learn inference	SQLite	GitHub REST API, optional Jira Cloud REST API, Joblib model/scaler, 120-second cache, automated email alerts

No common production database is currently established. Team Formation, Scheduling and Proposal Assessment use PostgreSQL within their own component boundaries, whereas the final Risk handoff documents SQLite persistence. This separation is consistent with the current integration evidence: components can be presented through the same IPMS application while their analytics and data stores remain independently owned.

5.2 Development and integration process

Development proceeded primarily through component-level research prototypes followed by interface and integration hardening. Each component was evaluated against evidence appropriate to its research question rather than through one common accuracy measure. Team Formation combined exact-search verification, controlled

synthetic experiments, a real-response case study and implementation checks. Scheduling combined saved/offline model experiments, controlled WBS fixtures, database-backed remediation tests and local cross-component captures. Risk Monitoring combined a synthetic classifier evaluation with live GitHub/Jira prototype exercises and alert delivery. Proposal Assessment documents its selected architecture and internal workflow, but its current backend test run did not pass



collection and its selected model performance values remain targets rather than achieved results.

Figure 5.1: Balanced Gantt schedule after LPT workload assignment in the scheduling component

The integration work follows an interface-first principle where evidence exists. Team Formation owns the official confirmed allocation and exposes it through `/api/final-allocations/active/teams` and supervisor-specific `/active/...` endpoints. The latest Scheduling evidence confirms a local HTTP 200 call from the Team Formation active-allocation API and successful rendering of returned teams in the scheduling team view. This demonstrates API consumption at the presentation boundary, but automatic mapping of the returned Team Formation identifiers into Scheduling project records has not yet been verified. Risk Monitoring and Proposal Assessment remain independently operating services within the common application; direct schedule/risk/quality payload exchange has not been demonstrated.

Table 5.2: Current implementation and integration evidence

Component	**Confirmed implementation / verification evidence**	**Current boundary**
Team Formation	Frontend lint and production build passed; backend compilation passed; allocation integrity, ACTIVE/ARCHIVED persistence, reporting, revision validation and REST provider APIs implemented	Formal usability study and full downstream backend import are not established

Scheduling	Owner-confirmed live LPT route; focused PostgreSQL remediation suite passed 6/6; hall-capacity checks passed for generation/reschedule/publication; Team Formation API-to-team-view capture returned HTTP 200	Automatic Team Formation-to-Scheduling backend ID mapping, direct Quality/Risk exchange, full fresh upload/webhook journey and some history/deadline edge cases remain open
Proposal Assessment	React/FastAPI/PostgreSQL architecture, `/analyze`, version/review logic and saved-review precondition are documented	Backend suite stopped during collection because `cachetools` was missing; achieved final model results, end-to-end finalisation and parent-IPMS integration remain unverified
Risk Monitoring	FastAPI prototype, GitHub/Jira retrieval, contribution/risk dashboard and automated HIGH RISK email delivery demonstrated; synthetic classifier evaluation documented	Direct cross-component data exchange, final API schemas, formal user validation and production deployment remain unverified

The available material does not yet provide a complete group-wide audit of sprint planning, branch/review policy, continuous-integration jobs, MS Planner/Teams activity or meeting records. Therefore, the common report should describe the evidence-backed component and integration workflow above without claiming a formal group process that has not been supplied.

5.3 Deployment and operational configuration

The current system should be described as a local research prototype rather than a production institutional deployment. Component services are run locally with their required application server and datastore. Team Formation, Scheduling and Proposal Assessment depend on reachable PostgreSQL services; the final Risk implementation documents SQLite persistence. FastAPI/Uvicorn-based services are used throughout the Python components, with browser-based interfaces communicating with their respective APIs.

Operational configuration is component-specific. Scheduling additionally requires a database URL, a student-submission integration key, GitHub webhook configuration for signed repository events, and access to any exposed endpoint used by GitHub. Proposal Assessment requires local model artefacts and PostgreSQL, with optional credentials for external LLM drafting and email delivery. Risk Monitoring requires GitHub credentials and optionally Jira configuration; its final handoff documents local application access and automated email alerts. Team Formation requires its cohort/project workbook inputs, optimisation configuration and PostgreSQL allocation store.

The prototype contains several useful reliability boundaries. Team Formation preserves previous confirmed allocations using ACTIVE/ARCHIVED versioning. Scheduling versions plans and stores document/repository evidence separately from approval decisions; its GitHub endpoint verifies HMAC signatures and its student portal uses a separate integration boundary. Proposal Assessment is designed to keep analysis and supervisor review associated with proposal versions and prevents finalisation when a required saved review is absent. Risk Monitoring can operate in GitHub-only mode when Jira is unavailable and uses a short-lived cache to reduce repeated external calls.

Table 5.3: Operational configuration and deployment boundaries

Area	**Current prototype position**	Requirement before institutional deployment

Runtime topology	Separate local component services and component-owned databases; common application/navigation available	Approved institutional hosting topology, stable domains/TLS, service discovery and operational ownership
Authentication/authorisation	Component-level role/access mechanisms exist in parts of the system; Scheduling and Proposal Assessment document role/ownership checks	One common authentication/SSO model and complete cross-component authorisation matrix
Secrets and external services	Database URLs, GitHub/Jira credentials, webhook secrets, integration keys and optional LLM/email credentials are component configuration	Central secret management, rotation policy and environment separation
Backup/recovery	Versioning and historical records exist in several components	Documented backup schedule, restore test, disaster-recovery objective and database/file retention policy
Observability	Local logs, test outputs, captured traces and component dashboards are available	Centralised logging, health checks, metrics, alerting and uptime/SLA targets

Security/privacy	Human review boundaries, restricted student actions and data-minimisation concerns are documented	Institutional privacy review, penetration/security testing, retention/deletion policy and deployment approval

Because these production controls are not yet established across the complete IPMS, this report should not claim production readiness, institutional security certification, guaranteed availability or disaster-recovery capability.

5.4 Evidence of team execution

Team execution is currently evidenced most strongly by component artefacts rather than a single consolidated project-management log. Each member has produced an individual research artefact and a common-report handoff that identifies the component problem, method, implementation, results, limitations and integration status. The common report uses these artefacts together with available code/test traces and screenshots to distinguish implemented work from intended or unverified work.

Table 5.4: Available evidence of team execution

Member / component	**Evidence currently available**	**Remaining evidence limitation / future validation**
Bandara N D M — Team Formation	Final individual thesis; exact-search and controlled evaluation results; scalability and real-response evidence; product/build checks; REST allocation interface and lifecycle evidence	Any additional downstream backend-consumption trace.

Lekamge R L M D — Scheduling	Individual dissertation plus newer corrected handoff; saved model/evaluation results; 6/6 focused PostgreSQL remediation test; hall-capacity tests; local screenshots/API trace including Team Formation team-view	A complete fresh student upload-to-review journey was not retained in the final evidence set. Automatic Team Formation backend ID mapping.
Rashmika R M D — Proposal Assessment	Individual thesis and latest handoff; documented architecture, datasets, model configuration, version/review workflow and contribution statement	Achieved final model experiment reports; successful backend rerun; exact finalisation API schemas; end-to-end runtime/integration evidence and UI screenshots
Kulathunga S A I K — Risk Monitoring	Synthetic model evaluation; GitHub/Jira prototype evidence; dashboard and delivered HIGH RISK email screenshot; contribution statement	Final endpoint schemas, AHP/expert records if claimed, direct cross-component payload evidence and formal user validation

This evidence is sufficient to establish that four substantial research components were developed and evaluated to different degrees, and that the common application brings them into one IPMS context. It is not yet sufficient to reconstruct a complete team-level execution history such as all commits, meetings, task assignments, review decisions and release milestones. Those records should be summarised rather than pasted into the final report if the group chooses to include them.

6 INTEGRATED TESTING EVALUATION AND VALIDATION

6.1 Evaluation framework and success criteria

The group-level evaluation treats IPMS as a set of specialised decision-support components rather than reducing the complete system to one accuracy score. Each component is evaluated with metrics appropriate to its function, while the common report checks whether the four stages collectively support a coherent undergraduate-project lifecycle. Evidence is classified as controlled/synthetic, historical real-world software data, real-response case-study data, local prototype/runtime evidence, or unavailable validation. This distinction is necessary because the components were not trained or tested on one shared dataset and their metrics are not directly comparable.

At system level, success requires more than strong component metrics. The common acceptance criteria are: (i) each component must expose a clearly bounded function and preserve human academic judgement; (ii) results must be traceable to the data and method that produced them; (iii) implemented interfaces must reject or report invalid states rather than silently fabricate data; (iv) verified cross-component exchanges must use stable identifiers and documented contracts; and (v) claims based on synthetic or prototype evidence must not be presented as institutional outcome evidence.

Table 6.1: Integrated evaluation framework and success criteria

Area	Primary evaluation question	Evidence / metrics	Success boundary
Team Formation	Can the system produce technically feasible and preference-aware team/project alternatives at practical scale?	Exact Pareto recovery; hypervolume; statistical tests; scalability; real-response case study; allocation integrity.	Strong optimisation evidence within the tested formulation; self-reported competency and one real-response cohort limit generalisation.
Adaptive Scheduling	Can proposals be converted into traceable plans, balanced assignments, reviewable evidence	WBS recovery; effort error; JIRA proxy classifier; workload CV; focused database tests; API/UI traces.	Planning and evidence workflow supported; proxy-model and historical-data

Area	Primary evaluation question	Evidence / metrics	Success boundary
	and feasible academic events?		results do not establish student assignment quality.
Proposal Assessment	Can proposal-analysis, retrieval and supervisor-review functions be implemented and evaluated without treating model output as an academic judgement?	Dataset-stage counts; selected classifier/grader targets; static workflow evidence; backend test status.	Architecture and workflow are documented. The selected classifier/grader numbers are performance targets rather than achieved final metrics, and complete backend runtime verification was not available in the final evidence set.
Risk & Contribution	Can execution activity be transformed into contribution/risk indicators and delivered through a usable monitoring prototype?	Synthetic classifier metrics; GitHub/Jira prototype tests; dashboard outputs; delivered HIGH RISK email alert.	Synthetic predictive performance is not evidence of real-project risk accuracy; human interpretation remains required.
Integrated IPMS	Can the four components operate as a cohesive lifecycle solution with verified interfaces where implemented?	Shared application access; Team Formation REST provider; Team Formation-to-Scheduling team-view trace; component-local runtime evidence.	Full automatic backend data exchange across all four components is not yet verified.

6.2 Integration and end to end testing

Integration testing was assessed at the level actually supported by retained evidence. The strongest cross-component evidence is the Team Formation provider interface and the Scheduling team-view consumer. A local capture returned HTTP 200 from the active-allocation endpoint and rendered the returned teams in the Scheduling interface. This demonstrates API consumption for display, but it does not prove automatic conversion of the returned Team Formation project/group identifiers into Scheduling database records.

Within Scheduling, the later remediation evidence is treated as authoritative over the earlier dissertation draft: one live LPT auto-assignment route was retained, and a focused PostgreSQL regression suite passed six of six tests covering ownership/versioning and hall-capacity checks for generation, rescheduling and publication. Other flows remain partially verified. In particular, a task-list call from the student portal was captured, but a fresh upload was not; a linked GitHub commit was visible, but a new signed webhook delivery was not captured during the same run.

Proposal Assessment has a documented internal upload-analysis-version-review-finalisation design, but its recorded backend test attempt stopped during collection because the cachetools dependency was unavailable. Therefore, neither the full component runtime path nor parent-IPMS exchange is counted as a passed integration test. Risk Monitoring has prototype evidence for GitHub/Jira retrieval, dashboard analysis and automatic email alert delivery, but direct exchange with the Scheduling or Proposal components is not established.

Table 6.2: Current integration and end-to-end verification status

Test / interface	Evidence	Status	Boundary
Team Formation active allocation API	GET /api/final-allocations/active/teams and ACTIVE/ARCHIVED lifecycle.	Verified provider	Confirms the current official allocation can be exposed without direct database coupling.
Team Formation -> Scheduling team view	Local HTTP 200 response and rendered team cards.	Verified for display	Automatic backend group/project-ID import into

Test / interface	Evidence	Status	Boundary
			Scheduling is not verified.
Scheduling live LPT assignment/versioning	Focused PostgreSQL regression suite: 6/6 passed after remediation.	Verified in isolated test DB	Not production or real-student outcome validation.
Scheduling hall-capacity checks	Generation, reschedule and publish regression checks passed.	Verified in focused suite	Institution-specific audience/equipment rules remain outside the test.
Student portal -> Scheduling	Task-list GET returned eligible tasks; existing submission visible.	Partially verified	Fresh end-to-end upload was not captured.
GitHub -> Scheduling progress	Previously linked commit and route evidence.	Partially verified	No new signed webhook delivery was captured in the final evidence run.
Proposal Assessment internal workflow	Routes, persistence and HTTP 409 saved-review precondition documented.	Backend suite did not complete collection in the supplied evidence; a complete successful runtime flow was not verified.	Backend suite did not complete collection; successful full runtime flow not available in the final evidence set.
Risk data -> dashboard / alert	GitHub/Jira prototype use and delivered HIGH RISK Gmail notification.	Prototype verified	Does not establish real-project predictive accuracy.
Four-component backend chain	No retained transaction showing all four	Not verified	Common portal access must not be

Test / interface	Evidence	Status	Boundary
	components exchange one project context automatically.		described as full data integration.

6.3 Overall results

The component results are summarised below without attempting to rank the four components against one another. Their datasets, outcomes and evaluation units are fundamentally different. Team Formation has the broadest optimisation validation, Scheduling combines historical software data with controlled fixtures and focused runtime checks, Risk Monitoring has a synthetic predictive experiment plus live external-API prototype evidence, and Proposal Assessment currently has a documented architecture and dataset pipeline but no verified final model-performance result.

Table 6.3: Synthesised component-level results

Component	Verified / reportable results	Evidence type	Interpretation and limitation
Team Formation	Exact 12-student verification enumerated 34,650 feasible allocations and recovered the verified medium/high-conflict Pareto fronts. Mean hypervolume = 0.9479 versus 0.6532 for unseeded NSGA-II (~45.1% higher). Friedman chi-square(3)=326.08, $p<0.001$; Kendall $W=0.906$. Synthetic 1,000-student/250-project pipeline ~30.15 s. Real-response case: f1=0, f2=0.05, 100% modelled technical coverage and 95% preference satisfaction.	Controlled exact + synthetic scalability + real-response case study	Strong evidence for the tested formulation and implementation. Competency is self-reported; the 60-student case is not institution-wide proof and f1=0 does not predict project success.
Adaptive Scheduling	JIRA proxy classifier: 81.27% held-out accuracy, weighted	Historical real-world software records + synthetic fixtures +	The classifier is a mixed JIRA outcome proxy,

Component	Verified / reportable results	Evidence type	Interpretation and limitation
	F1=0.815 on 1,411 issue records. WBS extractor recovered 73/75 explicit tasks in three synthetic proposals (97.3% recovery). Effort model log MAE=0.8554, log RMSE=1.0614; original-hour errors remain large. Offline workload CV: LPT 3.72%, guarded hybrid 4.40% across 100 trials. Later focused runtime suite: 6/6 passed.	offline simulation + isolated runtime tests	not student-suitability accuracy. Extraction result is fixture recovery, not general proposal accuracy. LPT was slightly fairer than the offline hybrid.
Proposal Assessment	Documented data stages: 2,227 raw annotations; 2,121 deduplicated spans; 2,226 raw feedback records; 2,151 embedded feedback records; 165 grading records on a 0-42 scale. Current MiniLM/Logistic Regression targets (0.850 accuracy, 0.800 macro-F1, 0.850 weighted-F1) and Ridge targets (MAE 1.50, RMSE 2.00, R2 0.50) are not achieved results.	Legacy dataset pipeline + design/implementation evidence	No final measured model performance should be claimed yet. Backend test collection was blocked by missing cachetools, and no formal user study or parent-IPMS execution is demonstrated.
Risk & Contribution	Synthetic dataset: 4,000 records, 3,200/800 train/test, 12 features. Logistic Regression accuracy 97.63% and weighted F1 ~0.98; Gradient Boosting 94.00%,	Synthetic held-out classification + prototype external-API/runtime evidence	High synthetic accuracy reflects prepared risk labels and must not be reported as real student-project risk accuracy. Real

Component	Verified / reportable results	Evidence type	Interpretation and limitation
	Random Forest 92.38%, Decision Tree 90.25%. Prototype exercised GitHub/Jira retrieval, dashboard analytics and automatic HIGH RISK email delivery.		GitHub/Jira use demonstrates operation, not independently labelled predictive validity.

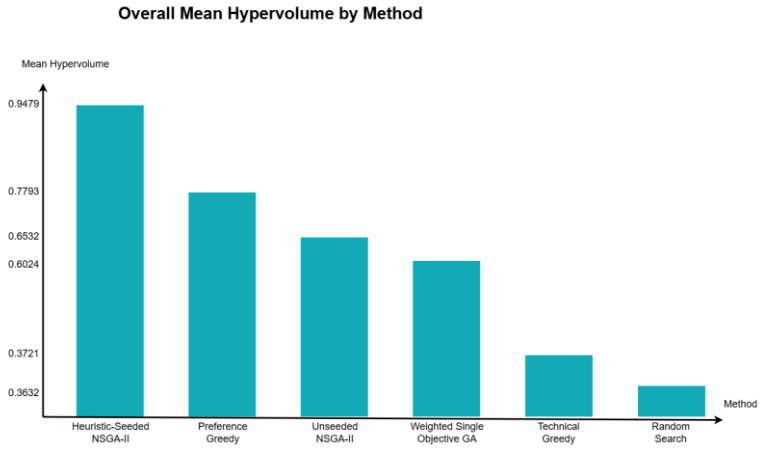


Figure 6.1: Overall mean hypervolume of the evaluated Team Formation allocation methods

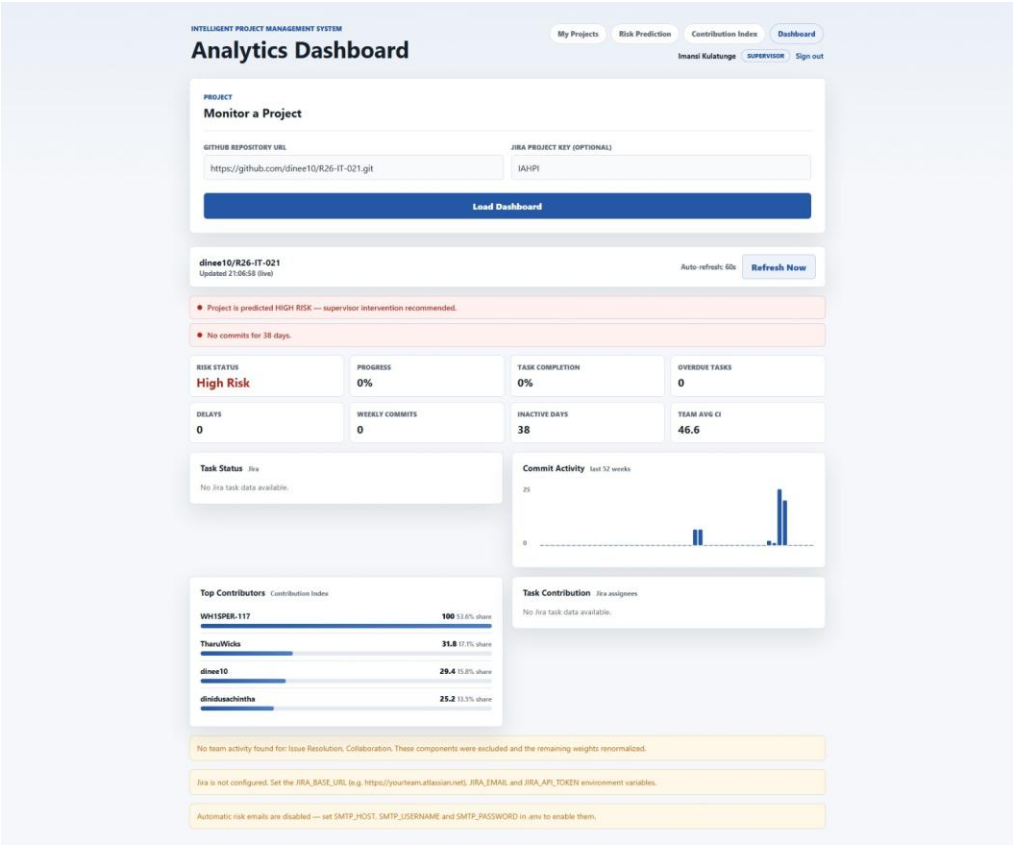


Figure 6.2: Predictive Risk Monitoring dashboard showing project risk and activity warnings

The group therefore does not report a single aggregate IPMS accuracy. The appropriate conclusion is that the prototype has multiple independently evaluated decision-support capabilities, but their evidence strength is uneven. In particular, Proposal Assessment still lacks final measured model results, and the complete four-component lifecycle has not been evaluated as one experimental system.

6.4 Stakeholder and user validation

Stakeholder evidence is currently component-specific rather than a formal evaluation of the complete IPMS. Team Formation includes the strongest direct academic validation: the technical requirements for the 15 projects used in the real-response case study were reviewed and approved before evaluation, and staff retain authority to select the final allocation from nondominated alternatives. This supports the validity of the project-requirement configuration used in that case, but it is not a usability study.

The Scheduling, Proposal Assessment and Risk/Contribution components do not currently provide a completed formal supervisor/student usability study or controlled educational-outcome study. Their interfaces and decision boundaries are designed around academic roles, and prototype evidence shows that major workflows can be presented to users, but no common-system evidence currently supports quantified claims of reduced staff workload, improved student learning, higher project completion rates or greater user satisfaction.

No formal common-system supervisor/student usability study or structured user-acceptance study was included in the final evidence set. Stakeholder evidence is therefore limited to component-level requirement review, demonstrations, interface/runtime evidence, and the human-review boundaries built into the components. The report does not claim measured complete-system usability, satisfaction, learning improvement, or administrative time savings.

6.5 Achievement of common objectives

Achievement of the common objectives was assessed against the final evidence set. “Achieved” is used only where the intended function and relevant evaluation are

both supported; “partially achieved” indicates that core functionality exists but important validation or integration evidence is unavailable or incomplete.

Table 6.4: Achievement of common group objectives

Objective	Current status	Evidence-based assessment
O1 — Feasible team/project allocation using technical requirements and student preferences	Achieved within evaluated scope	Separate technical/preference objectives, Pareto alternatives, exact verification, baseline comparison, scalability testing and a real-response case study are available.
O2 — Proposal-to-plan scheduling, workload assignment, evidence tracking and academic-event coordination	Substantially achieved; validation incomplete	Planning, versioning, evidence review and academic scheduling are implemented; live LPT assignment and hall-capacity remediation passed focused tests. Full external/end-to-end workflow and real-cohort outcomes remain unverified.
O3 — Advisory proposal analysis and adaptive mentorship with supervisor control	Partially achieved	Architecture, pipelines, persistence and human-review boundary are documented, but final measured ML results, complete backend runtime verification and stakeholder validation are missing.
O4 — Contribution and risk indicators from development/task activity	Achieved at prototype level	Contribution/risk analytics, synthetic classifier evaluation, GitHub/Jira operation, dashboard presentation and delivered email alert evidence are available. Real-project predictive validity remains unestablished.
O5 — Cohesive IPMS application with stable interfaces and shared identifiers where implemented	Partially achieved	Common application access exists. Team Formation provides a stable REST contract and Scheduling consumes it for team-view rendering. Automatic backend handoff and direct Proposal/Risk exchanges are not verified.
O6 — Evaluate components honestly and synthesise system evidence without overstating results	Partially achieved	Three components provide measured experimental/runtime results with clear boundaries; Proposal Assessment still lacks verified final performance and the whole system has no common

Objective	Current status	Evidence-based assessment
		user/outcome study.

6.6 Integrated limitations and threats

The principal threat to system-level interpretation is heterogeneity of evidence. The four components use different datasets, labels, evaluation units and maturity levels, so the common report cannot infer a single overall predictive accuracy or compare metric values as if they measured the same construct. Component-level results should be interpreted only within their stated data and task boundaries.

Integration remains incomplete. The Team Formation REST provider and Team Formation-to-Scheduling team-view consumption are verified, but automatic backend identity mapping into Scheduling records is not. Direct schedule/proposal/risk data exchange is also not established. Consequently, the system should be described as a partially integrated lifecycle prototype rather than a fully orchestrated four-service platform.

Data validity also differs by component. Team Formation relies on self-reported ordinal competencies and one 60-student real-response case study. Scheduling uses JIRA issue history as a proxy for student-work allocation and reports large effort-prediction errors; its extraction result uses deliberately structured synthetic proposals. Proposal Assessment has incomplete experimental verification and unresolved dataset-governance details. Risk Monitoring reports strong performance on synthetically generated labels, so generalisation to real undergraduate project risk remains unknown.

External validity is limited because no formal complete-system stakeholder study or institutional deployment has been completed. Security, privacy, identity management, backup/recovery, retention, licensing and production performance have not been verified uniformly across all services. These are deployment prerequisites rather than achievements of the current research prototype.

Finally, human oversight is an intentional safeguard but also means that algorithmic metrics alone do not establish educational value. A technically valid allocation, schedule, proposal warning or risk label can still require contextual academic

judgement. Future validation should therefore combine technical metrics with supervisor review quality, usability, decision time, student outcomes and longitudinal project evidence.

7 COMMERCIALISATION, SUSTAINABILITY AND RESPONSIBLE INNOVATION

7.1 Value proposition and beneficiaries

The integrated IPMS is intended to support the undergraduate project lifecycle from initial team and project allocation through planning, proposal review, evidence-aware execution monitoring and academic oversight. Its value proposition is not that any single algorithm replaces staff judgement, but that related decisions can be supported by traceable evidence within one project-management environment. Team Formation provides staff-reviewed team/project allocations; Scheduling converts project information into versioned plans and reviewable evidence; Proposal Assessment supports human-in-the-loop proposal analysis; and Risk Monitoring surfaces contribution and delivery indicators during execution.

The principal beneficiaries are academic staff, supervisors and undergraduate project teams. Academic staff benefit from structured allocation and scheduling support, supervisors receive planning, review and monitoring information, and students receive clearer project assignments, task expectations and feedback pathways. The component evaluations do not establish institution-wide time savings or improved learning outcomes, so these benefits should be presented as supported design intentions rather than measured organisational impact.

Table 7.1: Integrated value proposition and beneficiaries

Beneficiary	Current value offered by IPMS	Evidence boundary
Academic coordinators/staff	Team/project allocation, supervisor assignment, academic-event planning and traceable revision history.	Allocation and scheduling functions are implemented at prototype level; institution-wide administrative savings have not been measured.
Project supervisors	Source-linked plans, evidence review, proposal-analysis support, contribution/risk indicators and human confirmation points.	No formal complete-system usability study has been completed.

Beneficiary	Current value offered by IPMS	Evidence boundary
Students	Clearer team/project context, task/evidence submission paths, proposal feedback and visibility of monitored progress.	No controlled study demonstrates improved grades, learning or satisfaction.
Institution/department	A modular platform that can centralise multiple project-management support functions while preserving component ownership.	Production deployment, common identity management and institutional governance were not established in the final evidence set.

7.2 Market alternatives and differentiation

The supplied project evidence does not contain a formal market study, named-competitor benchmark or willingness-to-pay analysis. Consequently, this report does not claim commercial superiority over existing products. Differentiation is instead stated against functional substitute categories visible in the problem context: manual spreadsheets and email-based coordination, generic project-management tools that do not model academic allocation constraints, standalone proposal-analysis tools, and repository/activity dashboards that do not preserve the full academic decision trail.

The main functional differentiation is the combination of four academic-project functions under a common lifecycle: multi-objective team/project allocation, proposal-grounded planning and evidence tracking, supervisor-controlled proposal assessment, and contribution/risk monitoring. A further differentiator is the repeated use of human decision boundaries: staff select final team allocations, supervisors review generated plans and submitted evidence, proposal-analysis outputs remain advisory, and risk indicators support rather than replace academic judgement. These characteristics should be described as design differentiation, not as proven market advantage.

7.3 Business model and feasibility

The evidence currently supports technical feasibility at research-prototype level rather than a validated commercial business model. All four components have implementable software stacks and component-level test evidence, but the group has not produced a verified cost model, revenue model, customer-acquisition plan or production-service estimate. The most defensible near-term pathway is therefore an institutionally supervised pilot in which the system is evaluated with real academic workflows before any wider deployment or commercial decision is made.

Operational feasibility depends on resources already visible in the implementation: web frontends, FastAPI services, PostgreSQL or SQLite persistence, model artefacts, document storage, and external services such as GitHub, Jira and optional email/LLM providers. Additional production resources would include stable hosting, TLS, secret management, database backup and recovery, monitoring, user support and maintenance of external API integrations. No verified total-cost-of-ownership figure is available, so this report does not estimate deployment cost.

The business/sustainability model therefore remains a group-level decision. Before final submission, the report may be updated if the team chooses and justifies a specific institutional delivery model, but no subscription, licensing or revenue assumptions should be presented as established evidence without supporting analysis.

7.4 Adoption intellectual property and compliance

Adoption requires more than successful local execution. The current evidence shows uneven maturity across components and no common production identity, security or deployment framework. A real institutional deployment would therefore require authenticated role-based access, secure secret storage, documented backup and recovery, audit logging, retention and deletion rules, and clear ownership of shared identifiers and API contracts. These are adoption requirements; the current prototype should not be described as already compliant with an institutional security or privacy standard.

Privacy is particularly important because the platform may process student identifiers, self-reported competencies, proposal documents, supervisor comments, GitHub identities, submitted files and project activity. Data minimisation, purpose limitation and controlled access should be applied to each component. Optional external services introduce additional data-transfer considerations, and the proposal-assessment component still has unresolved dataset anonymisation, ethics and licence details. Risk and Scheduling components likewise require formal retention and institutional-use policies before handling production student data.

Intellectual-property and licensing readiness is also incomplete. The implementation uses multiple open-source frameworks, machine-learning libraries and external APIs, while some components additionally depend on third-party datasets or model artefacts. The group has not completed a consolidated licence and ownership audit. Before distribution beyond the research prototype, the team should verify software licences, model terms, dataset permissions, third-party API terms and ownership of the integrated codebase and research artefacts.

Responsible innovation is treated as a design requirement across the system. Self-reported competencies are not equivalent to verified ability; JIRA-derived classifier outputs are not student-suitability evidence; proposal scores and weakness predictions remain advisory; and synthetic risk-classification accuracy is not proof of real-student risk prediction. Human academic authority is therefore retained at critical decision points. Table 7.2 summarises the principal adoption controls still required.

Table 7.2: Responsible adoption and compliance readiness

Area	Current position	Required before institutional deployment
Identity and access	Role separation exists within components, but no verified common identity layer.	Common authentication/authorisation model, least-privilege roles and access testing.
Privacy and retention	Sensitive academic/project data are processed; policies are incomplete.	Data inventory, retention/deletion schedule, lawful/approved use, consent/notice where required.

Area	Current position	Required before institutional deployment
Security	Local prototypes use API keys/secrets and service-level controls.	TLS, managed secrets, hardened hosting, logging, backup/recovery and security testing.
Licensing/IP	Open-source libraries, models, datasets and external APIs are used.	Consolidated licence, dataset-rights and ownership audit.
Responsible AI	Human review boundaries are documented in all intelligent components.	Real-data validation, bias/error analysis, transparent explanations and escalation/override procedures.
Integration governance	Some interfaces are verified, but four-way automatic exchange is incomplete.	Approved shared identifiers, versioned contracts, failure handling and end-to-end acceptance tests.

7.5 Sustainable Development Goal contribution

The strongest common Sustainable Development Goal alignment is SDG 4, Quality Education. The intended contribution is indirect: IPMS aims to improve the structure, transparency and traceability of undergraduate project supervision by supporting team allocation, planning, proposal feedback, evidence review and early monitoring. These functions may help academic staff and students manage complex project activities more consistently, but the current research does not measure SDG-level educational impact.

A secondary alignment with SDG 9, Industry, Innovation and Infrastructure, is plausible because the project develops an integrated software platform and applies data-driven decision support to an educational process. However, this is a design-level association rather than demonstrated societal impact, and SDG 9 should remain secondary to the project's educational purpose.

Sustaining any SDG contribution would require more than maintaining the software. The system would need validated real-cohort outcomes, equitable access, appropriate privacy protection, trained staff, reliable infrastructure, ongoing model and API maintenance, and periodic review of whether automated indicators remain useful and fair. Possible negative effects - including over-reliance on model outputs, unfair

interpretation of self-reported skills or repository activity, and unnecessary collection of student data - should be actively monitored rather than treated as solved by automation.

8 CONCLUSION AND FUTURE DIRECTIONS

8.1 Overall conclusion

This common report presents the Intelligent Project Management System for Undergraduate Students as a lifecycle-oriented research prototype that addresses four connected project-management problems: forming and assigning teams, converting proposals into reviewable plans, supporting proposal quality assessment and mentorship, and monitoring contribution and delivery risk during execution. The evidence shows that each component can provide useful decision-support functions within its own boundary, while the complete system remains only partially integrated at the backend level.

The strongest verified system-level connection is the Team Formation allocation provider consumed by the Scheduling team-view interface. Other components are available within the same application context, but direct automatic data exchange among all four services has not been established. The evaluation therefore supports describing IPMS as a modular, partially integrated prototype rather than a fully orchestrated institutional platform.

Taken together, the results demonstrate feasibility rather than institutional effectiveness. Team Formation provides the most extensive optimisation and real-response evidence; Scheduling contributes measured offline and focused runtime results with explicit proxy limitations; Risk Monitoring demonstrates synthetic model performance and external GitHub/Jira operation; and Proposal Assessment documents a human-in-the-loop architecture whose final model and runtime verification remain incomplete. No common accuracy score or overall student-outcome improvement can be inferred from these heterogeneous results.

8.2 Principal integrated contribution

The principal integrated contribution is the design and partial implementation of a human-supervised undergraduate project-management environment that links allocation, planning, proposal review and execution monitoring while keeping the limitations of automated evidence visible. Rather than collapsing all decisions into

one model, IPMS assigns specialised analytical responsibilities to separate components and preserves staff or supervisor authority at the points where contextual academic judgement is required.

A second contribution is the use of traceable artefacts across the lifecycle: confirmed allocation versions, source-linked schedule tasks, submission and repository evidence, proposal-version review records, and dashboard risk/contribution indicators. Where implemented, versioning and stable interfaces allow a decision to be connected to the evidence from which it was derived. The common report also makes an explicit distinction between verified interfaces, portal-level navigation, component-local persistence and intended future cross-component flows, which reduces the risk of overstating integration maturity.

The integrated contribution is therefore architectural and process-oriented rather than a claim of a new universal optimisation or machine-learning algorithm. Its value lies in combining several decision-support functions around one undergraduate-project lifecycle and defining how human oversight, evidence provenance and component boundaries should be maintained as the system develops.

8.3 Future directions

Future work should first complete the technical integration contract among the four components. This includes agreeing stable group, project, member, supervisor and proposal identifiers; documenting versioned request/response schemas; validating automatic Team Formation-to-Scheduling backend mapping; and establishing whether Scheduling, Proposal Assessment and Risk Monitoring require direct data exchange or only shared project context. End-to-end tests should cover successful flows and failure recovery rather than relying only on portal navigation.

Second, component-level evidence gaps should be closed before stronger research claims are made. Proposal Assessment requires reproducible achieved model results, successful backend/runtime testing and verified review/finalisation paths. Scheduling should extend its focused regression evidence into a fresh end-to-end student submission and signed-webhook workflow and resolve remaining history/deadline edge cases. Risk Monitoring should be validated against independently labelled real

project outcomes and should reconcile any final AHP/expert evidence with the operational Contribution Index. Team Formation would benefit from larger multi-institution real cohorts, formal usability evaluation and further ablation of heuristic design choices.

Third, the complete system requires institutional-quality operational controls. A future deployment should provide common authentication and authorisation, secure secret management, privacy and retention policies, backup and recovery, audit logging, monitored service health, licence and dataset-rights review, and documented ownership of the integrated codebase. Production performance, concurrency and availability should then be tested under realistic workloads.

Finally, future evaluation should move beyond component metrics to educational and administrative outcomes. Structured supervisor and student studies could measure usability, decision time, quality of feedback, plan adherence, perceived fairness, rework and project outcomes while monitoring automation bias and unintended inequities. A formal market/adoption study and cost model would also be required before any commercialisation claim. These steps would determine whether the current research prototype can mature into a reliable institutional project-management platform.

REFERENCES

The following sources are cited in this common report.

[1] I. Taboada et al., “Artificial Intelligence Enabled Project Management: A Systematic Literature Review,” Applied Sciences, vol. 13, no. 8, art. 5014, 2023, doi: 10.3390/app13085014.

[2] M. K. Shaikh, “How to form a software engineering capstone team?” Heliyon, vol. 7, no. 4, Art. no. e06629, 2021.

[3] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in Proc. EMNLP-IJCNLP, 2019, pp. 3982–3992, doi: 10.18653/v1/D19-1410.

[4] E. Kalliamvakou et al., “The Promises and Perils of Mining GitHub,” in Proc. MSR, 2014, pp. 92–101, doi: 10.1145/2597073.2597074.

[5] K. Deb et al., “A fast and elitist multiobjective genetic algorithm: NSGA-II,” IEEE Trans. Evol. Comput., vol. 6, no. 2, pp. 182–197, 2002.

[6] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in Proc. ACM SIGKDD, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[7] R. L. Graham, “Bounds on Multiprocessing Timing Anomalies,” SIAM J. Appl. Math., vol. 17, no. 2, pp. 416–429, 1969, doi: 10.1137/0117039.

[8] S. Li and V. Ng, “Automated Essay Scoring: A Reflection on the State of the Art,” in Proc. EMNLP, 2024, pp. 17876–17888, doi: 10.18653/v1/2024.emnlp-main.991.

[9] T. L. Saaty, The Analytic Hierarchy Process. New York, NY, USA: McGraw-Hill, 1980.

APPENDICES

The appendices will contain only material needed to verify the integrated solution. Detailed component evidence remains in the individual theses and handoff packs.

Appendix A Integrated interface specification

Producer / boundary	Method / endpoint	Primary data / identifiers	Consumer / purpose	Verification status
Team Formation	GET /api/final-allocations/active/teams	allocation_id; revision_number; teams[]; group_key; project_id; members[]; supervisor	Scheduling team view and other downstream consumers	Implemented provider; local HTTP 200 and rendered Scheduling team-view trace captured
Team Formation	GET /api/final-allocations/active/supervisors/{supervisor_id}/groups	supervisor_id; current assigned groups	Supervisor-oriented consumers	Implemented provider endpoint
Scheduling	POST /api/schedule/upload-pdf	proposal PDF; group/project context	Proposal-to-plan workflow	Implemented component-local route
Scheduling	POST /api/schedule/generate	project parameters; extracted tasks; dates; team context	Versioned Gantt generation	Implemented component-local route
Scheduling	POST /api/progress/auto-assign	group/project; member IDs; persisted tasks	Authenticated live LPT ownership and balanced version	6/6 focused PostgreSQL remediation suite passed

Producer / boundary	Method / endpoint	Primary data / identifiers	Consumer / purpose	Verification status
Scheduling	GET /api/integrations/projects/{project_id}/document-tasks	project_id; eligible task/member context	Student submission portal	Task-list GET locally captured; a complete fresh upload journey was not retained in the final evidence set.
Scheduling	POST /api/integrations/document-submissions	project/task/member; document metadata/file	Student submission portal to evidence store	Route implemented; a fresh end-to-end upload trace was not retained in the final evidence set.
Scheduling	POST /api/webhooks/github	signed webhook body; repository; task key; actor mapping	Activity evidence and progress	Route implemented; a fresh signed delivery trace was not retained in the final evidence set.

Producer / boundary	Method / endpoint	Primary data / identifiers	Consumer / purpose	Verification status
Scheduling	POST /api/academic/auto-schedule	group/topic; event offsets/duration; staff/hall data	Academic timetable draft	Focused hall-capacity safeguards tested after remediation
Proposal Assessment	POST /analyze	proposal text; exact request schema not fully documented	Structural/ semantic proposal analysis	Documented route; complete runtime verification was not available in the final evidence set.
Proposal Assessment	Review-draft / finalisation routes	proposal version; supervisor review; final feedback	Human review and feedback generation	Review/finalisation behavior is documented; exact final URL/schema/authentication were not available in the final evidence set.

Producer / boundary	Method / endpoint	Primary data / identifiers	Consumer / purpose	Verification status
Risk Monitoring	GET /projects; POST /projects/upload; GET/POST /contribution; GET /dashboard; GET /api/dashboard-data; GET /api/jira-metrics; POST /predict; GET/POST /api/projects; POST /api/test-email	project/member context; GitHub repo; optional Jira key	Risk/contribution dashboard workflow	HTTP methods are documented in the final individual thesis; complete request/response schemas were not consolidated.
External services	GitHub REST / Jira Cloud REST	repository and task activity	Risk Monitoring; selected Scheduling evidence paths	Prototype integrations demonstrated; production policies remain local configuration

Table A.1: Integrated interface specification and verification status

Concept / identifier	Current component representations	Common-report rule
Student / member	Team Formation members[]; Scheduling member/student ID; Risk project-member context; Proposal Assessment IPMS-wide student field not confirmed	Shared concept only; do not claim one frozen field name/schema across all four components.
Team / group	Team Formation group_key/team_number; Scheduling group_id; Risk team/project context	Automatic group_key -> Scheduling group_id backend mapping is not verified.
Project	Team Formation project_id/project_title; Scheduling project/group record; Risk project/GitHub/Jira context; Proposal version context	Use project as the lifecycle join concept; a canonical cross-component project schema was not established.

Concept / identifier	Current component representations	Common-report rule
Supervisor	Team Formation supervisor_id/name; Scheduling supervisor context; Proposal review association; Risk user/dashboard context	Shared role concept exists; exact common identifier/authentication contract is not frozen.
Version / revision	Team Formation allocation revision; Scheduling schedule version; Proposal proposal/review version; Risk has separate project/risk records	Versioning is component-specific; do not imply one global version number.
Task / evidence	Scheduling task_id/task_key, submission_id, webhook/activity records; Risk contribution/activity records	No common task/evidence schema across Scheduling and Risk is verified.

Table A.2: Shared identifier concepts and current mapping boundaries

Appendix B Team contribution declaration

Member	Individual research / development contribution	Integration / shared contribution
Bandara N D M IT22114662	Multi-objective team/project formulation; Heuristic-Seeded NSGA-II V3; exact/baseline/scalability evaluation; staff workflow; validation; Technical Coverage Inspection; supervisor allocation; persistence and REST provider APIs.	Provides official team/project/supervisor allocation for downstream use; participated in common architecture, integration planning, documentation and presentation work.
Lekamge R L M D IT22117014	Proposal-grounded WBS extraction; effort/dependency-aware Gantt; live LPT task balancing; versioned evidence/review workflow; academic-event scheduling; evaluation and remediation testing.	Demonstrated Team Formation API consumption in Scheduling team view; participated in common architecture, integration planning, documentation and presentation work.
Rashmika R M D IT22101624	Semantic weakness analysis; advisory grading pipeline; historical-feedback retrieval; learning-resource recommendations; proposal-version/supervisor-review workflow; component research and implementation.	Candidate-provided integration contribution covers FastAPI exposure and the proposal/review workflow; no additional direct cross-component execution is claimed.
Kulathunga S A I K IT22109576	Contribution Index and weighting research; 12-feature risk classification; GitHub/Jira activity extraction; dashboard, risk inference and alert/email workflow; synthetic model evaluation.	Contributed the Risk component to the unified application and external activity integrations; direct four-component backend exchange is not implemented.

Table B.1: Team contribution declaration

Shared group activities: overall IPMS concept and lifecycle framing, common application/navigation decisions, integration discussions, progress-presentation

preparation, common-report preparation, and group-level demonstrations. No additional shared code, deployment, or testing ownership is assigned in this report where the supplied evidence does not establish it..